

EEF Meeting Minutes, Thursday June 23, 2022

Erlang Ecosystem Foundation, Minutes of a Regular Meeting of the Board of Directors Meeting #68, June 23, 2022

A regular meeting of the Erlang Ecosystem Foundation was held over group conference call (Zoom). The meeting began at 16:05 UTC.

Attendance

The following members of the board of directors were present (10/11): Sophie DeBenedetto, Francesco Cesarini, Maxim Fedorov, Kenneth Lundin, Ben Marx, Miriam Pena Villanueva, Peer Stritzinger, Sebastian Strollo, Frédéric Trottier-Hébert, and Alistair Woodman.

The following members of the board of directors were absent: Bryan Paxton

Also present was José Valim to present the EVision stipend request from Cocoa.

Agenda Items

1. Approval of minutes from previous meetings - item skipped as the previous minutes had not been sent out.
2. Intermediate actions: none to report
3. Financial update: \$190k in the bank with about \$15k in outstanding payments. Membership update: 140 annual supporting, 987 basic, 17 lifetime, 15 managing and contributing. New sponsor, Revelry, is asking for an invoice asap.
4. Unanimous consent votes: none scheduled
5. Stipend request. EVision (OpenCV) project from Cocoa. Presented by José Valim (who is acting sherpa).

RESOLVED, that the Erlang Ecosystem Foundation approve the request for \$20,000 to support the development of the EVision project as presented in the stipend request by . Unanimously approved.

6. Any other business: none.

7. Alistair declared the meeting adjourned at 16:30 UTC.

Dated June 23, 2022

Secretary of the meeting: _____ Sebastian Strollo

Attachments: EVision project presentation and stipend application (1 + 9 = 10 pages in total)

Stipend Type : Development Work
First name : Ruomeng
Last Name : Xu
Nick Name : Cocoa
Email Address : r.xu.1@research.gla.ac.uk
City : Glasgow
Country : United Kingdom
Twitter : @_uwu_cocoa
Linkedin : <https://www.linkedin.com/in/cocoa-xu/>
Website : <https://cocoa-research.works/>

Intended product of project :

A computer vision library that has at least a 50% code coverage rate in unit tests in Elixir with good integration with numerical-elixir (nx) and at least 3 use-case examples and documentation. (More details in section 2 of the attached application PDF)

Purpose

1. Bring computer vision into the world of Elixir via NIFs. 2. Remove the runtime dependency on some external languages. 3. Integration with numerical-elixir (nx). 4. Largely reduce the work of manually porting OpenCV functions/modules to Elixir. (More details in section 3 of the attached application PDF)

Development Plan

I will do the job for 12 months. The detailed stages for the development work are described in section 4 of the attached application PDF.

Dissemination : 1. Blog posts. The following series of blog posts on what this library can do and how to use it. 2. Conference talks. I was invited to give a talk at ElixirConf EU 2022. My supervisor, Dr Michala, will also present a keynote talk at the same conference on our group's research on federated learning in functional languages for IoT devices. We intend to participate in more BEAM related conferences in the 12 months. (More details for dissemination can be found in section 5 of the attached application PDF)

Delivery Method : This project is and will stay open-sourced on GitHub for everyone to access, use and contribute. We also intend to publish it on some Elixir package manager sites, e.g., hex.pm, after it is ready to ship.

Stipend Amount (USD) : 20000

Apply for the EEF stipends - Evision

Cocoa

Application

1. Stipend Type - Development Work

Evision¹ is a work in progress FOSS (Free Open-Source Software) licensed under Apache 2.0 on GitHub. This work was developed to support the applicant's PhD studies supervised by Dr. Anna Lito Michala and Prof. Phil Trinder at the University of Glasgow. The PhD project aims to develop resilient and fully distributed federated learning in Erlang/Elixir. However, to reach that goal it was necessary to develop the Evision library to enable more fine grained control of machine learning agent communication. Evision aims to largely reduce the work of manually porting OpenCV functions/modules to Elixir and provide native bindings between OpenCV and Elixir.

OpenCV is a library of programming functions mainly aimed at real-time computer vision. Originally developed by Intel, it was later supported by Willow Garage then Itseez. The library is cross-platform and free for use under the open-source Apache 2.0 License.

Currently, the initial findings of the project have attracted significant interest in the Elixir community, in particular, GitHub and Twitter. The statistic data on GitHub (Fig. 1) shows that we have more than 300 unique cloners and over 200 unique visitors, and that was the data for only the past 2 weeks.

¹ <https://github.com/cocoa-xu/evision>



Figure 1: Statistical data on GitHub for the evision project from 15th Jan to 28th Jan.

On Twitter, there are even more people showing a big interest in this project and would like to try it out. One of my tweet that demonstrates the potential of this library has gained over 15,000 impressions, 2,000+ engagements, 300+ likes and 35 retweets in less than 24 hours (link to the tweet).



Figure 2: Demonstrating objection detection with the *dnn* module using this library.

2. Intended product of the project

A computer vision library that has at least 50% code coverage rate in Elixir with good integration with numerical-elixir (nx) and at least 3 use-case examples and documentation. The documentation part includes

- (i) Transforming the original OpenCV inline docs to the format that Elixir uses.

At the moment of writting, *ex_doc* is the most widely use tools for generating HTML documentation from the source code. Therefore, this project will use *ex_doc* and ensure the transformed inline docs can be correctly displayed on the the generated HTML pages.

- (ii) Documentations on how to contribute to this project. For instance, how to integrate a new OpenCV module (either publicly available or an in-house one) into this project and make it available in Elixir.

We will provide an example of integrating a custom OpenCV module into this project.

Also, the library will have good integration with numerical-elixir (nx)² and other elixir libraries based on nx. It should be easy to use without explicitly converting data from Evision to nx and vice-versa. Meanwhile, I intend to accept feedback from the elixir machine learning community when designing the API.

² <https://github.com/elixir-nx/nx>

3. Purpose

- Bring computer vision into the world of Elixir via NIFs.

In some early test, I have observed that there is no performance loss, and evision on Elixir even runs slightly faster than the Python version ([link to the tweet](#)).

- Remove the runtime dependency on some external languages, e.g., Python.

In the past, if a user wants to use OpenCV in Elixir, then they usually install the Python version of OpenCV and open a *Port* in Elixir that talks to a Python process to interact with OpenCV.

This sometimes brings up the question that *why not just use Python if the things I interested in are mostly done in Python*. From my perspective, this could be a reason that refrains computer vision and/or machine learning users from embracing Erlang/Elixir.

- Integration with numerical-elixir (nx). Nx and machine learning libraries based on nx (such as Axon³) makes it much easier to do machine learning tasks. Particularly, computer vision is one of the important research areas in artificial intelligence.
- Largely reduce the work of manually porting OpenCV functions/modules to Elixir by using code to generate OpenCV-Elixir bindings.

³ <https://github.com/elixir-nx/axon>

In the past, there were some tries to port a few OpenCV functions to Elixir via NIFs, but, unfortunately, most (if not all) of them are no more maintained. For instance,

1. ConnorRigby/elixir-opencv. There are **11** OpenCV functions available. Last commit was on 30 Jan 2019, and it has been archived by the owner. It is now read-only.
2. zeam-vm/excv. There are **2** OpenCV functions available. Last commit was on 01 Dec 2021.
3. sriram-srinivasan/opencv-elixir. There are **6** OpenCV functions available. Last commit was on 05 Jul 2021.

Although it's possible to manually port OpenCV, the biggest problem is that it can take a few hours to a few days for porting and testing some functions.

For example, basic image displaying. OpenCV can display the image on screen (requires desktop environment and relevant libraries, e.g., GTK on Linux), however, native implemented functions are running in Erlang threads, and these threads are not the main thread therefore cannot perform UI (user interface) updates.

Therefore, I have to make proper adjustments to the code so that it could work.

Another thing that requires a few days or weeks of work is UI interaction. OpenCV supports adding UI control components to its window (that displays an image). For instance, users can add a slider to control the brightness of an image. This is easy to do in C++ as we can easily register a callback that listens to the changes from the slider. However, this is not something trivial to implement in this library.

There are over **4,000** functions, and this number is only for the main OpenCV repo (there is an *opencv-contrib* repo that contains more modules contributed by the community).

For comparison, evision has **3,961** OpenCV functions available at the moment. However, more tests should be written to test these functions. I'll get to this bit in the next section.

4. Development Plan

4.1 *Current Status* It covers most main OpenCV modules right now.

- (1) **calib3d**: Camera Calibration and 3D Reconstruction.
- (2) **core**: Core functionality.
- (3) **dnn**: Deep Neural Network module.
- (4) **features2d**: 2D Features Framework.
- (5) **flann**: Clustering and Search in Multi-Dimensional Spaces.
- (6) **imgcodecs**: Image Processing.
- (7) **imgproc**: Image file reading and writing.
- (8) **ml**: Machine Learning.
- (9) **photo**: Computational Photography.
- (10) **stitching**: Images stitching.
- (11) **ts**: Shape Distance and Matching.
- (12) **video**: Video Analysis.
- (13) **videoio**: Super Resolution.
- (14) **highgui**: High-level GUI. (Limited functionalities as a NIF library does not run in main thread).

There are already some demos and tutorials using this Evision library:

- (a) Philip Brown - Recognize Digits Using ML in Elixir.
- (b) RyoWakabayashi - Image processing using OpenCV (evision) with Elixir (move, rotate, filter, etc.).
- (c) Kentaro Kuribayashi - Beginning of evision-Image processing using OpenCV with Elixir.
- (d) Underjord - Elixir and Computer Vision.
- (e) Cocoa Xu - Evision + ncnn + MobileNetV3 SSDLite = Object Detection in Elixir - An Early-Stage Demo.

4.2 *Plan* Despite of the broad coverage of modules, it is still far from production level stability as it lacks of the following things:

- i *Integration with nx.* nx integration will be the first stage of this development work. As discussed above, there are machine learning libraries built on top of nx, and some tutorials are already using Evision and nx to show the ability of this library. Therefore, I can have more active feedback from the machine learning community and further improve this part.

This stage is estimated to take about 2-3 months of time.

- ii *Examples.* For second stage, I intend to write 10 more basic and 5 more complex examples showing what this library can achieve. This will attract more people who is interested in doing computer vision using Elixir. Moreover,
 - OpenCV provides some official examples, but they were written in other programming languages, which might not be quite helpful for Elixir users.
 - Also, Elixir users are expected to see examples in inline docs. We need to find a logical way to insert these crafted examples into the generated Elixir source code.
 - Furthermore, we can potentially have a *Livebook* version of these examples, which can be very easy for users to try this OpenCV-Elixir bindings.

This stage is estimated to take about 1.5 months of time.

- iii The third stage is to add support for modules in opencv-contrib. There are **about 60 modules** in the opencv-contrib repo. And a large portion of them need to be manually ported. Moreover, examples and documentation for them are also necessary. In the period of this proposal, I'd like to port around **20** modules from

opencv-contrib. Some of them are focus on practical uses, and some of them will focus on machine learning aspects.

One module of particular interest to the users is the **gapi** module. *gapi* stands for Graph API, which can be quite useful when feeding data through a computation graph that contains a number of computation nodes. For example, when inferring objects in a image using a neural network, however, not all nodes in that neural network will be used for inference (they may be only used in training stage). Graph API will automatically infer all the necessary internal nodes and do the computation to get the output, reducing the computation power needed.

Based on the documentation of OpenCV, possible real-life use-cases of this module can be

- (a) Face analytics pipeline
- (b) Porting anisotropic image segmentation
- (c) Face beautification algorithm

A tentative list is showing below⁴,

- (1) **barcode**: Barcode detecting and decoding methods.
- (2) **datasets**: Datasets Reader – Code for reading existing computer vision databases and samples of using the readers to train, test and run using that dataset's data.
- (3) **dnn_superres**: Superresolution using CNNs – Contains four trained convolutional neural networks to upscale images.
- (4) **face**: Face Recognition – Face recognition techniques: Eigen, Fisher and Local Binary Pattern Histograms LBPH methods.
- (5) **img_hash**: This module contains algorithms to extract hash of an image allowing to efficiently estimate similarity between images.
- (6) **optflow**: Optical Flow – Algorithms for running and evaluating deepflow, simpleflow, sparsotodenseflow and motion templates (silhouette flow).
- (7) **plot**: Plotting – The plot module allows you to easily plot data in 1D or 2D.
- (8) **quality**: Image Quality Analysis (IQA) API.
- (9) **rapid**: Silhouette based 3D object tracking.
- (10) **sfm**: Structure from Motion – This module contains algorithms to perform 3d reconstruction from 2d images. The core of the module is a light version of Libmv.
- (11) **shape**: Shape Distance and Matching.

⁴ Module name of their description are taken from the official OpenCV contrib repo, https://github.com/opencv/opencv_contrib

- (12) **structured_light**: Structured Light Use – How to generate and project gray code patterns and use them to find dense depth in a scene.
- (13) **superres**: Super Resolution
- (14) **text**: Scene Text Detection and Recognition – This module contains algorithms to perform text detection, words segmentation and text recognition in a visual scene.
- (15) **tracking**: Vision Based Object Tracking – Use and/or evaluate different visual object tracking techniques.
- (16) **videostab**: Video Stabilization.
- (17) **xfeatures2d**: Features2D extra – Extra 2D Features Framework containing experimental and non-free 2D feature detector/descriptor algorithms. SURF, BRIEF, Censure, Freak, LUCID, Daisy, Self-similar.
- (18) **ximgproc**: Extended Image Processing – Structured Forests / Domain Transform Filter / Guided Filter / Adaptive Manifold Filter / Joint Bilateral Filter / Superpixels / Ridge Detection Filter.
- (19) **xobjdetect**: Boosted 2D Object Detection – Uses a Wald-boost cascade and local binary patterns computed as integral features for 2D object detection.
- (20) **xphoto**: Extra Computational Photography – Additional photo processing algorithms: Color balance / Denoising / Inpainting.

This stage will take 8 months of time.

- iv Code coverage. A large number of tests need to be crafted and ran to ensure everything works as expected.

There are close to 4,000 functions at the moment. Although we don't need to write a test for each function (as some of them can be combined in a large single test), the number of tests can easily reach a few hundreds or more.

I cannot estimate the minimal number of tests that should be write. Instead, we should use the code coverage rate as the assessment for this part. The intended code coverage is at least 50%.

Note that these tests will not only cover the correct routes (where arguments are valid inputs), but also the bad ones. Moreover, functions in OpenCV often have a few options and these options should be tested as well.

- v Documentation.

- For users: there are plenty of inline docs in the OpenCV source code, and we hope a better support to automatically include these into generated Elixir code.

These docs, especially these math equations in the inline docs, will have better support in the help command, *h <function>*, as well as in the HTML/epub version generated by *ExDoc*.

- For contributors: a thorough guide of (a) how to add support for other OpenCV modules, (b) how to debug the auto-generated code of the NIFs.

vi Compatibility with future OpenCV versions, i.e., OpenCV 5.x. We currently support OpenCV 4.5.3, 4.5.4 and 4.5.5 (the latest version at the time of writing).

5. Dissemination

- Blog posts. The following series of blog posts on what this library can do and how to use it:
 - (i) How to install and setup everything. Ends with a demo showing basic operations like read and write an image.
 - (ii) How to integrate with Nerves, i.e., use Evision on embedded devices (e.g., Raspberry Pi). Ends with an example showing how to capture images from camera attached on the embedded device.
 - (iii) How to use the photo module from OpenCV to do computational photography. A possible tutorial is to demonstrate how to generate HDR photos in low-light conditions
 - (iv) Similar to the above one, I intend to write at least 2 blog posts for each supported OpenCV modules as tutorials or examples. Currently, we have supported 14 modules from the main OpenCV repo. Therefore, at least 28 blog posts. Meanwhile, once a module from the OpenCV-contrib is supported, I'll write at least 2 blog posts for that module.
- Conference talks. I was invited to give a talk at ElixirConf EU 2022⁵. Dr Michala will also present a keynote talk in the same conference on our group's research on federated learning in functional languages for IoT devices.

⁵ <https://www.elixirconf.eu/participants/cocoa-xu/>

The work from the PhD project using this library will lead to further conference talks. We aim to submit conference papers to the following tentative list of conferences:

- (i) ROSCon JP 2022. (19 October 2022)⁶. In this conference, we

⁶ <https://roscon.jp/>

intend to show how this project and Elixir can be used with ROS (The Robot Operating System) to achieve computer vision tasks collaboratively on at least two robots.

(ii) CodeBEAM American. (03-04 November 2022)⁷.

⁷ <https://codesync.global/conferences/code-beam-america-2022/>

These talks will cover the following points,

- i how we approached bringing OpenCV into Elixir
- ii at least one use-case evision demo for each talk
- iii how to integrate it with other Elixir libraries
- iv areas to focus for bringing machine vision to Elixir.

In one or more conference, we are particularly interested in showing how we can use CUDA (an OpenCV-contrib module that adds the support for using GPU to accelerate computations) to speed up the computer vision tasks in Elixir.

6. Delivery

This project is and will stay open-sourced on GitHub for everyone to access, use and contribute. We also intend to publish it on some Elixir package manager sites, e.g., *hex.pm*, after it is ready to ship.

7. Amount

20,000 USD for year 1 (12 months). This will cover the applicant's salary for 12 months to further develop the Evision library as part-time work alongside his full time PhD studies (Oct. 2021 - Oct. 2024). It should be paid directly to the applicant to avoid University of Glasgow overheads.

Dr Michala and Prof Trinder will not charge for their time as this work is already being supervised through the PhD supervision regular meetings and to avoid overhead charges.

No funds will be requested for equipment as access to Dr Michala's Accelerator Zoo equipment is already included as part of the PhD supervision offering.

The output will be instrumental to the continued work in Dr Michala's research group and will support further PhD researchers and research funding proposals (e.g. one EPSRC proposal already submitted in 2021).

The preferred payment method is wire transfer. The detailed information is shown below.

Full name: Ruomeng Xu

IBAN number: GB77NWBK52300625448285

BIC: NWBKGB2L

Sort-code: 52-30-06